

TrailGuard Project Brief

1. Introduction

TrailGuard is a field operations and wildlife protection system designed for Yala National Park. It supports conservation teams in managing patrols, recording incidents, handling conflict alerts, and generating reports in environments where internet connectivity is unreliable or unavailable.

The system is designed around an offline-first model, where field staff can continue working without a network connection and sync their data later when connectivity becomes available. This approach reflects the realities of remote conservation operations and ensures that field activity is captured reliably and accurately.

The project is grounded in the improved A02 design and addresses the shortcomings of the earlier A01 design by emphasizing data integrity, correct synchronization behavior, role-aware access, and trustworthy reporting.

2. Background and Problem Statement

Wildlife conservation and anti-poaching work often takes place in remote areas with limited or intermittent connectivity. Rangers and officers may need to:

- log patrol activity in the field
- capture incidents such as snares, wildlife hazards, and illegal activity
- record location and route information

- respond to alerts and conflict situations
- report operational outcomes and conservation findings

In many cases, a system that depends on live connectivity is impractical. If a field team cannot submit data in real time, the application must still allow them to record work locally and synchronize it later without losing data or creating misleading “submitted” states.

The major challenge is therefore not only having a functioning interface, but ensuring that the system behaves correctly under real-world constraints such as poor internet access, partial media uploads, duplicate submissions, and delayed synchronization.

3. Objectives

The primary objectives of TrailGuard are to:

- support offline-first field data collection
- ensure reliable synchronization of patrol and incident records
- improve alert assignment, acknowledgement, escalation, and closure
- maintain operational data integrity and user trust
- generate reports based only on synced, valid records
- enforce role-based access for different actors in the system

TrailGuard aims to balance practical field usability with technical reliability. The solution must remain useful in the field while also being correct, auditable, and consistent behind the scenes.

4. Scope

The system includes four main functional domains:

4.1 Patrol Management

- start and end patrols
- collect GPS and manual route points
- record location data while offline
- maintain active patrol state
- synchronize patrol records once connectivity is available

4.2 Incident Management

- create incident reports
- attach supporting evidence and media
- save local drafts and pending submissions
- retry failed uploads safely
- handle partial synchronization without duplication

4.3 Conflict and Alert Management

- create and assign alerts
- validate officer availability
- manage acknowledgements and outcomes
- escalate unresolved issues
- close alerts with final resolution information

4.4 Reporting and Analytics

- generate conservation reports
- enforce reporting windows and filters
- calculate route coverage metrics
- generate reviewable snapshots from synced records only
- export reports in standard formats

5. User Roles

5.1 Ranger

Rangers are frontline field operators. Their responsibilities include:

- starting and ending patrols
- recording route and waypoint data
- capturing incident reports and evidence
- reviewing local pending records
- retrying failed sync operations when connections return

5.2 Park Manager

Park managers are operational supervisors. Their duties include:

- reviewing alerts
- assigning officers

- escalating unresolved risk or conflict cases
- validating report generation outputs
- monitoring system status and field activity

5.3 Liaison Officer

Liaison officers coordinate operational response and outcomes. Their duties include:

- managing conflict closure
- validating final resolutions
- tracking action outcomes
- coordinating resources and field handoff

5.4 Researcher

Researchers use the system for analysis and reporting. Their responsibilities include:

- generating park-level reports
- analyzing patterns across synced records
- reviewing conservation metrics
- accessing approved report data

6. Functional Requirements

6.1 Patrol Management

- Patrols must be started with route information and assigned staff context.

- Location data must be recordable even during offline operation.
- Patrols must be saved locally as pending before synchronization.
- Duplicate active patrols must be prevented.
- Waypoints must be flushed correctly when a patrol ends.
- Failed patrol syncs must retry with backoff scheduling.

6.2 Incident Management

- Incident data must be captured offline.
- Validation should be possible before final submission.
- Local drafts and pending submissions must be supported.
- If a report is acknowledged but media upload fails, the system must preserve the incomplete state and retry safely.
- Already acknowledged content must not be re-sent unnecessarily.

6.3 Conflict Management

- Alerts must be created on fresh in-zone signals.
- Only one active assignment may exist per alert.
- Officer availability must be checked before assignment.
- Delivery state must be tracked independently from acknowledgement.
- If notification fails, availability must be restored.
- Unresolved cases must escalate after retry limits are exceeded.

6.4 Reporting

- Reports must be generated only from synced records.
- Reports must enforce a maximum 92-day reporting window.
- Coverage percentage must be computed using assigned route and covered distance.

- Pending records must be excluded from report snapshots.
- Empty report windows must produce a clear zero-state result.
- PDF and CSV exports must use the same snapshot ID for consistency.

7. Non-Functional Requirements

7.1 Offline-First Capability

- All writes must be local-first.
- Local state must remain visible and actionable without connectivity.
- The UI must not imply successful submission before acknowledgement is received.

7.2 Reliability and Consistency

- Idempotent upserts must be used for patrols, incidents, and reports.
- Retry logic must use exponential backoff with a maximum cap.
- Media uploads must only be marked complete after successful final receipt.
- Report generation must be deterministic and based on immutable snapshots.

7.3 Usability

- Offline and online status must be clearly visible.
- Pending queue and retry status must be visible.
- Minimal friction should be required for field data capture.
- The interface should support mobile-friendly interactions and larger touch targets.

7.4 Security and Access Control

- Navigation and actions must be role-aware.
- Permissions must be granted according to actor and workflow stage.
- Reporting and alert operations must be strictly tied to role permissions.

8. Proposed Architecture

8.1 Frontend

The frontend will be a modern web application using React and TypeScript with a route-based user flow for:

- patrol
- incidents
- conflict
- reports

The system will include:

- persistent local state for field data
- queue tracking for pending records
- role-switching UI
- offline and async state visibility

8.2 Domain Layer

The domain layer will include logic for:

- synchronization
- state transitions
- idempotency
- permissions and role rules
- report generation
- validation and status enforcement

This layer is separated from the interface to improve maintainability and reliability.

8.3 Data Layer

The data layer will manage:

- local pending records
- device-level storage
- synchronization metadata
- retry and backoff scheduling
- partial-upload tracking

8.4 Backend / Integration Layer

The backend will provide services for:

- patrol sync
- incident sync
- assignment processing
- notification delivery
- report generation

- acknowledgement tracking

9. Key Design Principles

The project is based on several critical design principles:

- local-first operation
- explicit pending and synced states
- role-aware workflows
- idempotent persistence
- retry-safe synchronization
- immutable report snapshots
- operational transparency for users

These principles are essential in preventing data loss, false success states, and inconsistent reporting in field conditions.

10. Risk Analysis

10.1 Connectivity Issues

Unstable connectivity may delay synchronization and create confusion between local and remote status. This is addressed using clear pending states and retry scheduling.

10.2 Partial Upload Failures

When media uploads partially fail, the system may incorrectly appear complete. This is mitigated by verifying final receipt before marking data as synced.

10.3 Duplicate Records

Duplicate submissions can corrupt the system if record identity is not handled carefully. Idempotent upserts and stable identifiers reduce this risk.

10.4 Assignment Conflicts

If multiple officers are assigned to the same alert, the operation may become inconsistent. Enforcement of single active assignment rules prevents this.

10.5 Reporting Inconsistency

Reports can become untrustworthy if generated from non-deterministic or mutable data. Immutable snapshots ensure consistency and auditability.

11. Implementation Roadmap

Phase 1 — Core Domain and Persistence

- define domain models and enums
- establish local storage and pending state handling
- implement sync contracts and idempotent operations
- build patrol and incident workflows

Phase 2 — Conflict and Assignment Workflows

- implement alert creation and assignment
- manage acknowledgement and escalation
- enforce single active assignment rules
- support closure workflows

Phase 3 — Reporting and Analytics

- define reporting criteria and validation rules
- generate synced-only snapshots
- compute coverage and reporting metrics
- add export functions

Phase 4 — UI Refinement and Testing

- improve pending queue visibility
- strengthen role-based navigation
- run domain and integration tests
- validate edge cases and state transitions

12. Acceptance Criteria

The system is considered successful when:

- patrol data can be recorded and synchronized offline-first
- incident reports survive offline conditions and retry correctly
- conflict alerts can be assigned, acknowledged, escalated, and closed

- reports are generated consistently from synced records only
- the UI clearly reflects pending, failed, synced, and escalated states
- role-based permissions are enforced correctly
- the solution passes the required domain validation and testing plan

13. Deliverables

The completed project will include:

- functional TrailGuard web application
- domain logic and data model
- offline-first synchronization and retry features
- role-aware user interface and route structure
- report generation and export functionality
- project documentation and test coverage report

14. Conclusion

TrailGuard is a practical field operations platform designed for law enforcement and conservation management in remote environments. It focuses on functionality, reliability, and correctness in difficult field conditions rather than only on interface appearance.

The improved A02 concept transforms the original idea into a resilient, realistic solution that supports real-world patrol operations, incident handling, conflict management, and trustworthy reporting. This makes the project suitable as both a technical implementation and a strong example of reliable offline-first system design.

If you want, I can also turn this into:

- a more academic assignment-format version,
- a shorter executive brief,
- or a version with section numbering and formal report styling for direct saving as a final project brief.